LAPORAN PRATIKUM 5
PEMOGRAMAN BEROENTAS OBJEK



Disusun Oleh:
Silva Nurifvan
2511533004

Dosen Pengampu :
Nurfiah, S.ST, M.Kom
Asisten Praktikum :
Siti Fadhilah Rahmi

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pembuatan sistem perbankan, setiap jenis rekening memiliki ketentuan transaksi yang berbeda-beda. Sebagai contoh, rekening tabungan membatasi penarikan berdasarkan jumlah saldo yang tersedia, sedangkan rekening giro dapat melakukan penarikan melebihi saldo dengan batas overdraft yang telah ditentukan. Perbedaan aturan tersebut membutuhkan mekanisme yang memungkinkan program menjalankan proses transaksi sesuai dengan jenis rekening yang digunakan.

Salah satu konsep dalam Pemrograman Berorientasi Objek yang dapat digunakan untuk menangani perbedaan tersebut adalah *Polymorphism* atau polimorfisme. Konsep ini memungkinkan suatu method dengan nama yang sama menghasilkan perilaku yang berbeda bergantung pada objek yang menggunakannya. Pada praktikum ini, penerapan polimorfisme dilakukan melalui *Method Overriding* pada method tarikTunai() di kelas RekeningGiro, serta penggunaan *Dynamic Method Dispatch* untuk menentukan method yang dijalankan ketika program dieksekusi. Selain itu, konsep polimorfisme juga diterapkan dalam pengembangan fitur transfer antar rekening.

1.2 Tujuan

1. Memahami konsep polimorfisme dalam Pemrograman Berorientasi Objek.
2. Mengetahui penerapan *Method Overriding* pada kelas turunan.
3. Memahami mekanisme *Dynamic Method Dispatch* pada bahasa pemrograman Java.
4. Menggunakan anotasi @Override dalam proses penimpaan method.
5. Mengembangkan fitur transaksi yang bekerja sesuai dengan jenis rekening.
6. Menerapkan konsep polimorfisme dalam pembuatan fitur transfer antar rekening.

1.3 Manfaat

1. Menambah pemahaman mengenai penggunaan polimorfisme dalam pemrograman Java.
2. Mengurangi penggunaan percabangan untuk mengatur perbedaan perilaku antarobjek.
3. Memudahkan pengembangan program yang memiliki beragam ketentuan transaksi.
4. Memperdalam pemahaman tentang *Method Overriding* dan *Dynamic Method Dispatch*.
5. Melatih keterampilan dalam membangun fitur transaksi perbankan dengan menerapkan konsep Pemrograman Berorientasi Objek.

BAB II

PEMBAHASAN

2.1 Teori

Polimorfisme merupakan salah satu konsep dalam Pemrograman Berorientasi Objek yang memungkinkan sebuah method memiliki perilaku yang berbeda sesuai dengan jenis objek yang menjalankannya. Pada sistem perbankan, konsep ini dapat digunakan dalam proses penarikan tunai karena rekening biasa dan rekening giro memiliki ketentuan penarikan yang berbeda.

1. **Polimorfisme (*Polymorphism*)**

   Polimorfisme adalah konsep yang memungkinkan suatu method memiliki beberapa bentuk perilaku. Dalam bahasa Java, pemanggilan method yang sama dapat menghasilkan proses yang berbeda bergantung pada jenis objek yang digunakan saat program berjalan. Contohnya, method tarikTunai() dapat menerapkan aturan yang berbeda pada rekening biasa dan rekening giro.

2. **Method Overriding**

   Method Overriding merupakan proses ketika kelas turunan membuat implementasi baru dari method milik kelas induk dengan nama dan parameter yang sesuai. Pada praktikum ini, kelas RekeningGiro melakukan penimpaan terhadap method tarikTunai() agar rekening giro dapat melakukan penarikan hingga batas overdraft yang telah ditentukan.

3. **Dynamic Method Dispatch**

   Dynamic Method Dispatch adalah mekanisme dalam Java yang menentukan implementasi method berdasarkan jenis objek sebenarnya ketika program dijalankan. Melalui mekanisme ini, pemanggilan akunAktif.tarikTunai() dapat menjalankan method dari kelas Rekening atau RekeningGiro sesuai dengan objek yang digunakan, tanpa memerlukan percabangan khusus untuk membedakan jenis rekening.

4. **Anotasi @Override**

   Anotasi @Override berfungsi sebagai penanda bahwa sebuah method pada kelas turunan dibuat untuk menimpa method yang terdapat pada kelas induk. Penggunaan anotasi ini membantu compiler menemukan kesalahan, misalnya apabila nama atau parameter method tidak sesuai dengan method yang ingin ditimpa.

5. **Penerapan Polimorfisme pada Fitur Transfer**

Polimorfisme dapat digunakan dalam pengembangan fitur transfer antar rekening melalui method transfer(Rekening tujuan, double nominal, String pinInput). Parameter Rekening tujuan memungkinkan method menerima objek rekening dari berbagai kelas turunan. Proses transfer mencakup pemeriksaan PIN, penarikan saldo dari rekening pengirim, serta penyetoran dana ke rekening penerima apabila proses penarikan berhasil.

2.2 Langkah-Langkah

Praktikum 5 merupakan kelanjutan dari praktikum 1 hingga praktikum 4. Pada tahap ini, program yang telah dibuat sebelumnya dikembangkan dengan menambahkan dan menyesuaikan beberapa bagian pada class tertentu. Class yang digunakan masih sama seperti praktikum sebelumnya, yaitu Transaksi, Rekening, RekeningTabungan, RekeningGiro, dan Main.

Adapun perubahan yang dilakukan pada praktikum ini akan dijelaskan pada bagian berikut.

1. Membuat Class Rekening

Gunakan kembali kode program dari praktikum 4, yaitu prak4_Inheritance, kemudian lakukan perubahan pada bagian method tarikTunai(). Perubahan ini bertujuan untuk menyesuaikan implementasi program dengan konsep polimorfisme yang dipelajari pada praktikum 5.

Berikut merupakan screenshot kode program setelah dilakukan perubahan.

```
public boolean tarikTunai(double nominal) {
    if (nominal <= 0) {
        System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
        return false;
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi, Saldo Anda: Rp" + saldo);
        return false;
    } else {
        saldo -= nominal;

        String idTrx = "TRX-T-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Debit", nominal));

        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Sisa saldo: Rp" + saldo);
        return true; // transaksi sukses
    }
}
```

Kode tarikTunai() digunakan untuk melakukan penarikan uang dari rekening. Method ini mengg unakan tipe data booleanuntuk menunjukkan apakah transaksi berhasil atau gagal, sedangkan par ameter double nominal digunakan untuk menentukan jumlah uang yang ingin ditarik.

Pada bagian if (nominal <= 0), program memeriksa apakah nominal penarikan kurang dari atau s ama dengan nol. Jika kondisi terpenuhi, program menampilkan pesan kegagalan dan mengembali kan nilai false. Selanjutnya, bagian else if (nominal > saldo) memeriksa apakah jumlah uang yan g ditarik melebihi saldo yang tersedia. Jika saldo tidak mencukupi, transaksi dibatalkan.

Apabila nominal penarikan valid dan saldo mencukupi, program menjalankan bagian else untuk mengurangi saldo sesuai jumlah uang yang ditarik. Kemudian, program membuat ID transaksi m enggunakan System.currentTimeMillis() dan menyimpan data penarikan ke dalam riwayatTransa ksi dengan jenis transaksi "Debit".

Terakhir, program menampilkan pesan bahwa penarikan berhasil beserta sisa saldo rekening. Nil ai return true menandakan bahwa transaksi berhasil dilakukan. Dengan demikian, method ini me mbantu memastikan proses penarikan berjalan sesuai ketentuan sekaligus mencatat transaksi ke d alam riwayat rekening.

2. Class RekeningGiro

```java
package pekan5;

import java.text.NumberFormat;

public class RekeningGiro extends Rekening {

    private double batasOverdraft;

    // Constructor
    public RekeningGiro(
            String nomor,
            String nama,
            double saldoAwal,
            String pinAwal,
            double batasOverdraft) {

        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

    // Getter batas overdraft
    public double getBatasOverdraft() {
        return batasOverdraft;
    }

    // Override method tarikTunai
    @Override
    public boolean tarikTunai(double nominal) {
        if (nominal <= 0) {
            System.out.println("Gagal: Nominal tarik harus lebih dari 0!");
            return false;
        } else if (nominal > saldo + batasOverdraft) {
            System.out.println("Transaksi Gagal: Melebihi batas overdraft! Maksimal penarikan: Rp"
                    + (saldo + batasOverdraft));
            return false;
        } else {
            saldo -= nominal;

            String idTrx = "TRX-G-" + System.currentTimeMillis();
            riwayatTransaksi.add(
                    new Transaksi(idTrx, "Debit Giro", nominal));

            NumberFormat rupiah = NumberFormat.getCurrencyInstance(
                    new java.util.Locale("id", "ID"));

            System.out.println("Tarik tunai " + rupiah.format(nominal) + " berhasil.");
            System.out.println("Saldo saat ini: " + rupiah.format(saldo));
```

Kode program tersebut merupakan implementasi kelas RekeningGiro yang merupakan turunan d ari kelas Rekening. Kelas ini memiliki atribut batasOverdraft yang digunakan untuk menentukan batas penarikan uang melebihi saldo yang tersedia. Pada bagian constructor, parameter nomor re kening, nama pemilik, saldo awal, dan PIN diteruskan ke kelas induk menggunakan super(), seda ngkan nilai batas overdraft disimpan melalui this.batasOverdraft. Method getBatasOverdraft()dig unakan untuk mengambil nilai batas overdraft rekening giro.

Pada bagian tarikTunai(), terdapat anotasi @Override yang menandakan bahwa method tersebut menimpa method milik kelas induk. Program terlebih dahulu memeriksa apakah nominal penarik an kurang dari atau sama dengan nol. Jika iya, transaksi ditolak. Selanjutnya, program memeriks a apakah nominal penarikan melebihi jumlah saldo ditambah batas overdraft. Jika melebihi batas tersebut, program menampilkan pesan kegagalan. Namun, jika penarikan memenuhi ketentuan, s

aldo akan dikurangi sesuai nominal yang ditarik, termasuk memungkinkan saldo menjadi negatif selama masih dalam batas overdraft.

Setelah penarikan berhasil, program membuat ID transaksi dengan awalan TRX-G-
, kemudian menyimpan data transaksi bertipe "Debit Giro" ke dalam riwayatTransaksi. Class Nu
mberFormat digunakan untuk menampilkan nominal uang dalam format mata uang rupiah agar h
asilnya lebih mudah dibaca. Terakhir, program menampilkan jumlah penarikan dan saldo terkini,
kemudian mengembalikan nilai true sebagai tanda bahwa transaksi berhasil.

Dengan demikian, kelas RekeningGiro menerapkan konsep polimorfisme melalui Method Overri
ding, yaitu mengubah perilaku method tarikTunai() dari kelas induk agar sesuai dengan aturan re
kening giro yang memperbolehkan penarikan melebihi saldo hingga batas overdraft tertentu.

Output

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
0.Keluar
Pilih Menu:
1
Masukan No Rekening: 0322796623
Masukan Nama Pemilik: silva
Masukan Saldo Awal: 100000
Masukan PIN(6 digit):310107

Piih Produk:
1.Tabungan Umum
2.Giro Bisnis
Piih Produk: 2
Masukan Batas Overdraft:
50000
Rekening atas nama silva berhasil dibuat.
Rekening Giro Berhasil dibuat.
```

Pilih 1, untuk membuka rekening baru dan masukan saldo awal sebesar Rp 100.000.
Pilih Produk 2, yaitu Giro Bisnis masukan batas Overdraftnya Rp 50.000

LATIHAN

1. Buat method Transfer di class Rekening, tambahkan satu method bar dengan nama : public void transfer(Rekening tujuan, double nominal, String pinInput)
   • Method ini menerima parameter polimorfik Rekening tujuan, bisa menerima Tabungan atau Giro.
   • Logika Method
     ▪ Pertama, lakukan otentikasi(pinInput). Jika false, batalkan.

▪ Kedua, panggil this.tarikTunai(nominal). (overriding, jika this adalah Giro, ia otomatis menggunakan aturan overdraft).

▪ Ketiga, periksa kembalian nilai tarikTunai. Jika true, panggil tujuan.setorTunai(nominal).

2. Integrasi ke CLI Utama: Buka Main.java dan tambahkan menu 8. Transfer ke Rekening Lain.
   • Minta pengguna memasukkan Nomor Rekening Tujuan.
   • Cari nomor tersebut di dalam daftarRekening.
   • Jika ketemu, minta nominal transfer dan PIN, lalu panggil akunAktif.transfer(akunTujuan, nominal, pin).

Jawab:

1. Method transfer di class Rekening

```
public Rekening(String nomor, String nama, double saldoAwal, String pinAwal) {
    this.nomorRekening = nomor;
    this.namaPemilik = nama;
    this.saldo = saldoAwal;

    //validasi PIN dalam Constructor
    if (pinAwal.length() == 6) {
        this.pin =pinAwal;
    } else {
    System.out.println("Peringatan: PIN harus 6 digit Menggunakan PIN default 123456");
    this.pin = "123456";
    }
```

Kode tersebut merupakan method transfer() yang digunakan untuk melakukan transfer ua ng dari satu rekening ke rekening lain. Method ini menerima tiga parameter, yaitu Rekeni ng tujuan sebagai rekening penerima, double nominal sebagai jumlah uang yang akan ditr ansfer, dan String pinInput sebagai PIN yang dimasukkan oleh pengguna.

Pada bagian if (!otentikasi(pinInput)), program memeriksa apakah PIN yang dimasukkan sesuai dengan PIN rekening pengirim. Jika PIN salah, program menampilkan pesan "PIN salah!" dan menghentikan proses transfer menggunakan return.

Selanjutnya, program menjalankan this.tarikTunai(nominal) untuk menarik uang dari rek ening pengirim. Jika penarikan berhasil, program menjalankan tujuan.setorTunai(nominal) untuk memasukkan uang ke rekening penerima. Setelah itu, pesan "Transfer berhasil!" d itampilkan sebagai tanda bahwa proses transfer telah dilakukan.

Dengan demikian, method ini menerapkan konsep Pemrograman Berorientasi Objek deng an memanfaatkan method dari objek rekening pengirim dan penerima. Pemanggilan this.t arikTunai(nominal) juga mendukung polimorfisme, karena method penarikan yang dijala nkan dapat berbeda sesuai jenis rekening pengirim, misalnya Rekening atau RekeningGir o.

2. Menu Main

```java
case 8:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        System.out.print("Masukkan Nomor Rekening Tujuan: ");
        String nomorTujuan = input.nextLine();

        Rekening akunTujuan = null;

        for (Rekening rekening : daftarRekening) {
            if (rekening.getNomorRekening().equals(nomorTujuan)) {
                akunTujuan = rekening;
                break;
            }
        }

        if (akunTujuan == null) {
            System.out.println("Rekening tujuan tidak ditemukan!");
        } else if (akunTujuan == akunAktif) {
            System.out.println("Tidak dapat transfer ke rekening sendiri!");
        } else {
            System.out.print("Masukkan nominal transfer: ");
            double nominal = input.nextDouble();
            input.nextLine();

            System.out.print("Masukkan PIN: ");
            String pinTransfer = input.nextLine();

            akunAktif.transfer(akunTujuan, nominal, pinTransfer);
        }
    }
    break;
```

Kode tersebut merupakan bagian `case 8` pada menu program perbankan mini yang digunakan un tuk menjalankan fitur transfer uang antar rekening. Program terlebih dahulu memeriksa apakah p engguna sudah memiliki rekening aktif. Jika `akunAktif` bernilai `null`, program menampilkan pe san bahwa pengguna belum membuka rekening. Jika rekening aktif tersedia, program meminta p engguna memasukkan nomor rekening tujuan.

Selanjutnya, program membuat variabel `akunTujuan` dengan nilai awal `null`, kemudian menggu nakan perulangan `for` untuk mencari rekening yang nomor rekeningnya sesuai dengan input pen gguna di dalam `daftarRekening`. Jika rekening ditemukan, objek rekening tersebut disimpan ke dalam `akunTujuan` dan perulangan dihentikan menggunakan `break`.

Setelah pencarian selesai, program memeriksa apakah rekening tujuan ditemukan. Jika `akunTuju an` masih bernilai `null`, program menampilkan pesan bahwa rekening tujuan tidak ditemukan. Ji ka rekening tujuan ternyata sama dengan `akunAktif`, program menolak transfer ke rekening sen diri.

Apabila rekening tujuan valid dan berbeda dari rekening aktif, program meminta pengguna mem asukkan nominal transfer menggunakan `input.nextDouble()`. Setelah itu, `input.nextLine()` digunakan untuk membaca sisa karakter baris agar input berikutnya tidak terlewat. Program kem udian meminta PIN pengguna untuk proses autentikasi.

Terakhir, perintah `akunAktif.transfer(akunTujuan, nominal, pinTransfer)` menjalankan method `transfer()` untuk memproses transaksi. Method tersebut menerima objek rekening tuju an, jumlah uang yang ditransfer, dan PIN yang dimasukkan pengguna. Proses transfer selanjutny a bergantung pada validasi PIN dan keberhasilan penarikan saldo dari rekening pengirim.

Dengan demikian, kode ini membantu memastikan transfer hanya diproses jika pengguna memili ki rekening aktif, rekening tujuan ditemukan, dan rekening tujuan bukan rekening milik pengirim sendiri.

Output

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
8.Transfer ke Rekening Lain
0.Keluar
Pilih Menu:
1
Masukan No Rekening: 0322796623
Masukan Nama Pemilik: cipaucantik
Masukan Saldo Awal: 100000
Masukan PIN(6 digit):310107

Piih Produk:
1.Tabungan Umum
2.Giro Bisnis
Piih Produk: 2
Masukan Batas Overdraft:
500000
Rekening atas nama cipaucantik berhasil dibuat.
Rekening Giro Berhasil dibuat.

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
8.Transfer ke Rekening Lain
0.Keluar
Pilih Menu:
8
Masukkan Nomor Rekening Tujuan: 0322796615
Rekening tujuan tidak ditemukan!
```

Link yt : https://youtu.be/DBOYEl5SRIU?si=BoWxdt75cdA9GkEB